

透過 Managed Service for Apache Spark 和 Lightning Engine 加快 Spark 速度

來源：<https://codelabs.developers.google.com/codelabs/lightning-engine?hl=zh-tw>

步驟數：8

使用 Serverless for Apache Spark Standard 和 Premium 方案搭配 Lightning Engine，比較 Spark 執行圖表和執行階段。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [準備環境](#)
4. [執行基準無伺服器工作](#)
5. [使用 Serverless Premium 和 Lightning Engine 執行](#)
6. [在 Spark UI 中比較執行圖](#)
7. [清除所用資源](#)
8. [恭喜](#)

1. 簡介

在本程式碼研究室中，您將探索 [Managed Service for Apache Spark](#) 原生執行引擎 [Lightning Engine](#) 的效能優勢，並瞭解這項引擎如何將 **Managed Apache Spark** 無伺服器上的 Spark 工作負載效能提升 4.3 倍。

Lightning Engine 使用 [Velox](#) 和 [Apache Gluten](#)。Velox 是用於資料處理的高效能 C++ 引擎。Apache Gluten 是中間層，負責將以 JVM 為基礎的 Spark 工作轉換為可由 Velox 執行的 C++ 程式碼。

本示範使用 [TPC-DS](#)，這是業界標準的基準，旨在評估決策支援系統的效能。您將提交基準 PySpark 工作，使用標準無伺服器層級查詢範例 TPC-DS 資料集。接著，您會使用啟用 Lightning Engine 的 Premium 層級，執行完全相同的工作。最後，您會比較執行時間，並深入瞭解 Spark UI，以視覺化方式呈現硬體加速 Spark 執行圖表的差異。

假設您按照「[清除](#)」一節所述，及時清除資源，執行本程式碼研究室的預估費用不到 **\$1.00 美元**。

學習內容

- 建立 [Cloud Storage](#) bucket，儲存基準測試指令碼和結果
- 使用代管 **Apache Spark** 無伺服器標準層，執行基準 PySpark 資料處理工作
- 使用 **Managed Apache Spark** 無伺服器 **Premium** 層級和 Lightning Engine 執行相同工作
- 比較執行階段指標
- 啟動 Spark 記錄伺服器 UI，比較原生實體執行圖

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
 - 已啟用計費功能的 [Google Cloud 專案](#)
 - 熟悉 [Apache Spark](#) 和 Linux 指令列的基本操作
-

2. 事前準備

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：



```
gcloud auth list
```

3. 確認專案已設定完成：



```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：



```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

執行下列指令，啟用本程式碼研究室的所有必要 API：

```
gcloud services enable \
  dataproc.googleapis.com \
  storage.googleapis.com \
  compute.googleapis.com
```

步驟 3 · 約 5 分鐘

3. 準備環境

在這個步驟中，您將初始化環境變數並建立 Cloud Storage bucket。這個 bucket 會存放您提交至兩個 Serverless for Apache Spark 層級的 PySpark 指令碼。

設定環境變數

在 Cloud Shell 執行下列指令，設定預設環境變數。我們將使用 `us-central1` 區域，但您可以視需要變更。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1"
export BUCKET_NAME="spark-benchmark-${PROJECT_ID}-${REGION}"

gcloud config set dataproc/region ${REGION}
```

建立 Cloud Storage bucket

建立 bucket 來存放指令碼和記錄：

```
gcloud storage buckets create gs://${BUCKET_NAME} \
  --uniform-bucket-level-access \
  --location=${REGION}
```

將 TPC-DS 資料集複製到自己的值區

在本步驟中，您會將 TPC-DS 資料集從公開 bucket 複製到自己的 Cloud Storage bucket。確保 PySpark 工作可從專案本機讀取資料。

設定環境變數，選擇資料集大小和類型：

```
export DATASET_TYPE="partitioned" # Options: partitioned, nonpartitioned
export DATASET_SIZE="1GB"          # Options: 1GB, 10GB, 100GB, 1000GB (1000GB not available
for partitioned)

export SRC_PATH="gs://beam-tpcds/datasets/parquet/${DATASET_TYPE}/${DATASET_SIZE}"
export DATASET_PATH="gs://${BUCKET_NAME}/tpc-ds-dataset/${DATASET_TYPE}/${DATASET_SIZE}"
```

費用警告：執行這些工作所需的時間和費用會隨著資料集大小增加。

將 TPC-DS 資料複製到自己的 bucket：

```
gcloud storage cp -r ${SRC_PATH}/* ${DATASET_PATH}/
```

建立 PySpark 基準指令碼

我們將使用 PySpark 指令碼，從 Cloud Storage 值區註冊標準 TPC-DS 資料表，並執行從 Apache Spark 公開存放區取得的 5 個標準查詢。指令碼會將資料集路徑做為引數。

在 Cloud Shell 中建立名為 `benchmark.py` 的檔案。您可以複製並貼上下列指令來產生檔案：

```

cat << 'EOF' > benchmark.py
import argparse
import sys
from pyspark.sql import SparkSession
import time

def main():
    parser = argparse.ArgumentParser(description='TPC-DS Benchmark')
    parser.add_argument('data_path', help='GCS base path for TPC-DS tables')
    args = parser.parse_args()

    base_path = args.data_path

    # Initialize Spark Session
    spark = SparkSession.builder \
        .appName("TPC-DS Benchmark") \
        .getOrCreate()

    print(f"Spark Session created. Registering TPC-DS tables from {base_path}...")

    # List of all 24 TPC-DS tables
    tables = [
        "call_center", "catalog_page", "catalog_returns", "catalog_sales",
        "customer", "customer_address", "customer_demographics", "date_dim",
        "household_demographics", "income_band", "inventory", "item",
        "promotion", "reason", "ship_mode", "store", "store_returns",
        "store_sales", "time_dim", "warehouse", "web_page", "web_returns",
        "web_sales", "web_site"
    ]

    # Register each table as a temporary view
    # For this subset of queries, not every table is used
    for table in tables:
        path = f"{base_path}/{table}"
        try:
            df = spark.read.parquet(path)
            df.createOrReplaceTempView(table)
        except Exception as e:
            print(f"Warning: Could not load table {table} from {path}. Error: {e}")

    print("Tables registered successfully. Starting benchmark queries from Apache Spark test suite...")

    # Standard TPC-DS Queries sourced from Apache Spark public repository:
    # https://github.com/apache/spark/tree/master/sql/core/src/test/resources/tpcds
    queries = {
        "Q1": """
            WITH customer_total_return AS (
                SELECT sr_customer_sk AS ctr_customer_sk,
                       sr_store_sk AS ctr_store_sk,
                       sum(sr_return_amt) AS ctr_total_return
                FROM store_returns, date_dim
        """
    }

```

```

        WHERE sr_returned_date_sk = d_date_sk
        AND d_year = 2000
        GROUP BY sr_customer_sk, sr_store_sk
    )
    SELECT c_customer_id
    FROM customer_total_return ctr1, store, customer
    WHERE ctr1.ctr_total_return > (
        SELECT avg(ctr_total_return) * 1.2
        FROM customer_total_return ctr2
        WHERE ctr1.ctr_store_sk = ctr2.ctr_store_sk
    )
    AND s_store_sk = ctr1.ctr_store_sk
    AND s_state = 'TN'
    AND ctr1.ctr_customer_sk = c_customer_sk
    ORDER BY c_customer_id
    LIMIT 100
    """,
    "Q2": ""
    WITH wscs AS (
        SELECT sold_date_sk, sales_price
        FROM (
            SELECT ws_sold_date_sk AS sold_date_sk, ws_ext_sales_price AS sales_price
            FROM web_sales
            UNION ALL
            SELECT cs_sold_date_sk AS sold_date_sk, cs_ext_sales_price AS sales_price
            FROM catalog_sales
        )
    ),
    wswscs AS (
        SELECT d_week_seq,
            sum(CASE WHEN (d_day_name='Sunday') THEN sales_price ELSE null END) AS
sun_sales,
            sum(CASE WHEN (d_day_name='Monday') THEN sales_price ELSE null END) AS
mon_sales,
            sum(CASE WHEN (d_day_name='Tuesday') THEN sales_price ELSE null END) AS
tue_sales,
            sum(CASE WHEN (d_day_name='Wednesday') THEN sales_price ELSE null END)
AS wed_sales,
            sum(CASE WHEN (d_day_name='Thursday') THEN sales_price ELSE null END) AS
thu_sales,
            sum(CASE WHEN (d_day_name='Friday') THEN sales_price ELSE null END) AS
fri_sales,
            sum(CASE WHEN (d_day_name='Saturday') THEN sales_price ELSE null END) AS
sat_sales
        FROM wscs, date_dim
        WHERE d_date_sk = sold_date_sk
        GROUP BY d_week_seq
    )
    SELECT d_week_seq1,
        round(sun_sales1/sun_sales2, 2),
        round(mon_sales1/mon_sales2, 2),
        round(tue_sales1/tue_sales2, 2),

```



```

        round(wed_sales1/wed_sales2, 2),
        round(thu_sales1/thu_sales2, 2),
        round(fri_sales1/fri_sales2, 2),
        round(sat_sales1/sat_sales2, 2)
FROM (
    SELECT wswscs.d_week_seq AS d_week_seq1,
           sun_sales AS sun_sales1, mon_sales AS mon_sales1,
           tue_sales AS tue_sales1, wed_sales AS wed_sales1,
           thu_sales AS thu_sales1, fri_sales AS fri_sales1,
           sat_sales AS sat_sales1
    FROM wswscs, date_dim
    WHERE date_dim.d_week_seq = wswscs.d_week_seq
          AND d_year = 2001
) Y,
(
    SELECT wswscs.d_week_seq AS d_week_seq2,
           sun_sales AS sun_sales2, mon_sales AS mon_sales2,
           tue_sales AS tue_sales2, wed_sales AS wed_sales2,
           thu_sales AS thu_sales2, fri_sales AS fri_sales2,
           sat_sales AS sat_sales2
    FROM wswscs, date_dim
    WHERE date_dim.d_week_seq = wswscs.d_week_seq
          AND d_year = 2001 + 1
) Z
WHERE d_week_seq1 = d_week_seq2 - 53
ORDER BY d_week_seq1
""",
"Q3": ""
SELECT dt.d_year,
       item.i_brand_id AS brand_id,
       item.i_brand AS brand,
       sum(ss_ext_sales_price) AS sum_agg
FROM date_dim dt,
     store_sales,
     item
WHERE dt.d_date_sk = store_sales.ss_sold_date_sk
      AND store_sales.ss_item_sk = item.i_item_sk
      AND item.i_manufact_id = 436
      AND dt.d_moy = 12
GROUP BY dt.d_year,
         item.i_brand,
         item.i_brand_id
ORDER BY dt.d_year,
         sum_agg DESC,
         brand_id
LIMIT 100
""",
"Q7": ""
SELECT i_item_id,
       avg(ss_quantity) AS agg1,
       avg(ss_list_price) AS agg2,
       avg(ss_coupon_amt) AS agg3,

```

```

        avg(ss_sales_price) AS agg4
FROM store_sales,
     customer_demographics,
     date_dim,
     item,
     promotion
WHERE ss_sold_date_sk = d_date_sk
      AND ss_item_sk = i_item_sk
      AND ss_cdemo_sk = cd_demo_sk
      AND ss_promo_sk = p_promo_sk
      AND cd_gender = 'M'
      AND cd_marital_status = 'S'
      AND cd_education_status = 'College'
      AND (p_channel_email = 'N' OR p_channel_event = 'N')
      AND d_year = 2000
GROUP BY i_item_id
ORDER BY i_item_id
LIMIT 100

```

""",

"Q19": ""

```

SELECT i_item_id,
       i_brand,
       i_category,
       i_class,
       i_manufact,
       sum(ss_ext_sales_price) AS sales,
       sum(ss_net_profit) AS profit

```

```

FROM date_dim,
     store_sales,
     item,
     customer,
     store

```

```

WHERE d_date_sk = ss_sold_date_sk
      AND i_item_sk = ss_item_sk
      AND d_year = 2000
      AND d_moy = 12
      AND c_customer_sk = ss_customer_sk
      AND s_store_sk = ss_store_sk
      AND i_manager_id = 9

```

```

GROUP BY i_item_id,
         i_brand,
         i_category,
         i_class,
         i_manufact

```

```

ORDER BY i_item_id,
         i_brand,
         i_category,
         i_class,
         i_manufact

```

```

LIMIT 100

```

""

}

```

total_start_time = time.time()

for query_name, query_sql in queries.items():
    print(f"\nExecuting {query_name}...")
    query_start = time.time()

    # Execute query and force action using show()
    result_df = spark.sql(query_sql)
    result_df.show(5) # Show top 5 rows

    query_end = time.time()
    print(f"{query_name} completed in {query_end - query_start:.2f} seconds.")

total_end_time = time.time()
print(f"\nAll benchmark queries completed in {total_end_time - total_start_time:.2f}
seconds.")

spark.stop()

if __name__ == "__main__":
    main()
EOF

```

將指令碼複製到 Cloud Storage 值區，讓 Serverless for Apache Spark 存取：

```
gcloud storage cp benchmark.py gs://{BUCKET_NAME}/scripts/benchmark.py
```

4. 執行基準無伺服器工作

如要提供不含 Lightning Engine 的基準比較，請將先前上傳的 PySpark 基準化工作提交至 Serverless for Apache Spark Standard 層級。我們會將您複製的資料集路徑做為引數傳遞。

執行下列指令來執行批次工作：

```
gcloud dataproc batches submit pyspark \  
  gs://${BUCKET_NAME}/scripts/benchmark.py \  
  --region=${REGION} \  
  --version=2.3 \  
  --deps-bucket=gs://${BUCKET_NAME} \  
  -- ${DATASET_PATH}
```

注意：首次提交無伺服器工作時，Serverless for Apache Spark 可能需要 1 到 2 分鐘來暖機後端基礎架構，工作才會實際開始執行。`benchmark.py` 計時器只會在 Spark 啟動後，測量實際處理時間。

監控工作

工作執行期間，您會在 Cloud Shell 終端機中看到 PySpark 記錄串流。Serverless for Apache Spark 會分配容器、從 Cloud Storage 讀取 TPC-DS Parquet 資料集，並執行複雜的 SQL 計畫。

指令碼執行完畢後，請觀察控制台輸出內容。您應該會看到每個執行的標準查詢結果和時間，類似於：

```
...  
Executing Q1...  
+-----+  
|c_customer_id|  
+-----+  
...  
  
Q1 completed in 18.52 seconds.  
...  
  
All benchmark queries completed in 110.94 seconds.
```

請記下完成作業的總秒數。這是您的**基準執行階段**。

5. 使用 Serverless Premium 和 Lightning Engine 執行

接著，您會在代管 **Apache Spark** 無伺服器上執行完全相同的 Spark 工作，但會使用進階級，並啟用 Google 原生的向量化查詢引擎：**Lightning Engine**。

Serverless for Apache Spark 會完全抽象化叢集佈建作業。您只需為工作執行時間付費。

提交基準工作至 Serverless，並明確啟用 Lightning Engine：

```
gcloud dataproc batches submit pyspark \  
  gs://${BUCKET_NAME}/scripts/benchmark.py \  
  --region=${REGION} \  
  --version=2.3 \  
  --deps-bucket=gs://${BUCKET_NAME} \  
  --properties="dataproc.tier=premium,spark.dataproc.lightningEngine.runtime=native" \  
  -- ${DATASET_PATH}
```

比較結果

等待工作完成並檢查輸出內容。您應該會看到相同的查詢結果。請仔細查看完成時間：

```
...  
All benchmark queries completed in 64.24 seconds.
```

比較基準作業與 Lightning Engine 作業後，您會發現 Lightning Engine 運用原生 C++ 執行層和後端的向量化處理，可加快分組、彙整和聯結作業的執行速度，而且完全不需要變更 PySpark 應用程式程式碼。

工作負載越大，Lightning Engine 的效能就越好。在本例中，我們使用的是小型資料集，因此效能提升幅度可能不如預期。在 10 TB 的資料集上，基準測試顯示效能較開放原始碼 Spark [提升 4.3 倍](#)。

6. 在 Spark UI 中比較執行圖

執行時間縮短的幅度令人驚豔，但讓我們深入瞭解 Spark 在查詢執行期間實際執行的作業。您可以檢查這兩項工作的 Spark UI 執行圖表，瞭解這點。

1. 在瀏覽器中開啟 [Google Cloud 控制台](#)。
2. 依序前往「Managed Apache Spark」>「Batches」。
3. 清單中會顯示兩批資料：標準基準執行作業和進階級執行作業。
4. 按一下您執行的 Premium 層級批次，然後依序點選「View Spark UI」(查看 Spark UI) 和「View Details」(查看詳細資料)。
5. 在 Spark UI 中，前往「Jobs」分頁。
6. 在「已完成的工作」下方的搜尋框中輸入 `velox`。
7. 您會看到許多包含 `VeloxSparkPlanExecApi` 的職位說明。這是指 Lightning Engine 使用的 Velox 原生執行引擎。

現在，請針對標準層級執行重複這個程序：

1. 返回 Serverless for Apache Spark 的「Batches」(批次) 頁面。
 2. 按一下「Standard tier」(標準層級) 批次的連結，然後依序點選「View Spark UI」(查看 Spark UI) 和「View Details」(查看詳細資料)。
 3. 在 Spark UI 中，前往「Jobs」分頁。
 4. 在「已完成的工作」下方的搜尋框中輸入 `velox`。
 5. 工作說明中不會提及 Velox API。
-

步驟 7 · 約 2 分鐘

7. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收費，請刪除本程式碼研究室建立的資源。

在 Cloud Shell 中，刪除 Cloud Storage bucket 和當中內容：

```
gcloud storage rm -r gs://${BUCKET_NAME}
```

刪除 `benchmark.py` 的本機副本：

```
rm benchmark.py
```

提示：受管理的 Apache Spark 批次工作不會留下運算基礎架構。系統只會針對批次工作實際執行的時間向您收費。

8. 恭喜

恭喜！您已成功為 Apache Spark 建構基準化環境，並比較 Managed Apache Spark 無伺服器標準版與 Managed Apache Spark 無伺服器進階版。

您親眼見證啟用 Managed Apache Spark 無伺服器的新 Lightning Engine 後，Spark 工作負載的執行階段如何縮短，並探索 Spark UI，瞭解實體執行圖表如何使用 Native Query Engine 轉換為原生 C++ 程式碼。

目前所學內容

- 如何編寫 PySpark 資料集基準測試指令碼。
- 如何將 Spark 工作提交至 Managed Apache Spark 無伺服器。
- 如何啟用 Lightning Engine。
- 如何在 Spark UI 中比較工作方案。

後續步驟

- 探索 [Managed Apache Spark 無伺服器說明文件](#)
 - 請參閱[原生查詢執行資格工具](#)
 - 在 [GitHub](#) 上查看完整的 TPC-DS 基準化查詢
-